Frontier Atlas — Architecture

Goal. Build a source-traceable AI intelligence ingestion platform capable of a theoretical 500k+ records while preserving freshness, deduplication, deterministic entity resolution, and auditable provenance.

1. End-to-end flow

Source discovery → async crawlers → raw immutable observation layer → URL/date/content validation → full-text cleaning → intelligent chunking → multi-tier LLM extraction → Pydantic validation → deterministic entity resolution → PostgreSQL primary store → Redis queue/cache → Google Sheets export. Optional graph/vector systems are secondary projections, not the source of truth.

Scale model. Acquisition is page/cursor based and checkpointed. Workers use bounded concurrency, connection pooling and backpressure. Raw payloads are persisted before expensive enrichment so failures resume without re-crawling everything.

2. Reliability and provider orchestration

413 handling. Estimate token size before requests. Split on sentence/paragraph boundaries with overlap. If a provider still returns HTTP 413, halve the chunk budget and retry only the affected work. Never resend the whole document unnecessarily.

429 handling. Honor Retry-After when supplied; otherwise use exponential backoff with jitter. Retry transient 408/425/429/5xx responses. Concurrency limits prevent a retry storm.

LLM tiers. Gemini Flash is first, then Groq Llama, then DeepSeek. Provider-specific adapters share a small interface. Extraction must return schema-valid JSON. If a provider fails repeatedly, the orchestrator falls through to the next provider. API keys are never stored in source control.

Freshness. A single UTC now value is used per ingestion run. Relative timestamps are normalized; records older than 24 hours, future-dated records, or records without trustworthy dates are excluded from the Phase II final dataset.

3. Distributed freshness and deduplication

Canonical URL removes fragments, default ports and common tracking parameters. Content SHA-256 detects identical bodies. A uniqueness key on (record_type, source_url) makes entity persistence idempotent. Content hashes remain non-unique because two legitimate publishers can contain identical text. Acquisition checkpoints make page/cursor progress resumable across workers.

4. Storage choices and access strategy

Layer	Role	Why
PostgreSQL	Canonical entities, provenance, checkpoints	ACID transactions, constraints, JSON payloads, mature indexing
Redis	Queue/cache/rate-limit coordination	Fast ephemeral coordination and backpressure
Google Sheets	Evaluator-facing export	Simple auditable six-tab deliverable
Vector DB (optional)	Semantic retrieval projection	Useful for embeddings, but not authoritative
Graph DB (optional)	Relationship projection	Useful for company/product/paper/repo links at larger scale

Compliance. Crawlers identify themselves, respect robots.txt/terms/access controls, use public APIs/feeds where available, and may use Playwright for legitimately accessible JS-rendered pages. CAPTCHA, Cloudflare/Datadome challenges and other access controls are not bypassed.

Observability. Emit per-source counts, latency, status codes, retry counts, freshness rejection counts and dedup counts. Every final record retains source URL and collection timestamp. This enables audit and safe reprocessing.